

1. Linear Regression

Model & Prediction

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b \quad (\text{compact: absorb } b \text{ into } \mathbf{w})$$

Loss: Mean Squared Error (MSE)

$$\mathcal{L}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (y_i - \mathbf{w}^\top \mathbf{x}_i)^2$$

Normal Equation (Closed-Form)

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Requires $\mathbf{X}^\top \mathbf{X}$ invertible. Cost: $O(d^3 + nd^2)$.

Gradient Descent

Gradient of MSE:

$$\nabla_{\mathbf{w}} \mathcal{L} = -\frac{2}{n} \mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w})$$

Update rule (η = learning rate):

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L}$$

- **Batch GD**: all n samples per step (stable, slow)
- **SGD**: 1 random sample per step (fast, noisy)
- **Mini-batch**: batch of B samples (best trade-off)

2. Regularization

Why Regularize?

Prevent overfitting by penalising large weights:

$$\mathcal{L}_{\text{reg}} = \underbrace{\mathcal{L}_{\text{data}}}_{\text{fit data}} + \underbrace{\lambda R(\mathbf{w})}_{\text{keep weights small}}$$

$\lambda > 0$ controls regularisation strength.

Ridge Regression (L2)

$$\mathcal{L} = \frac{1}{n} \sum (y_i - \hat{y}_i)^2 + \lambda \|\mathbf{w}\|_2^2$$

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$$

Shrinks all weights; none become exactly zero.
Geometric constraint: **circle/hypersphere**.

Lasso Regression (L1)

$$\mathcal{L} = \frac{1}{n} \sum (y_i - \hat{y}_i)^2 + \lambda \|\mathbf{w}\|_1$$

Drives some weights *exactly to zero* $\Rightarrow$ feature selection.

Geometric constraint: **diamond**.

L1 vs L2 Quick Comparison

Property	L1 (Lasso)	L2 (Ridge)
Penalty	$\lambda \sum w_j $	$\lambda \sum w_j^2$
Sparsity	Yes (weights = 0)	No
Feat. select	Built-in	Not directly
Shape	Diamond	Circle
Differentiable	No (at 0)	Yes

Bias-Variance Trade-off

$$\text{Total Error} \approx \text{Bias}^2 + \text{Variance} + \text{Noise}$$

- **High bias** (underfitting): model too simple
- **High variance** (overfitting): model too complex
- Regularisation: slightly $\uparrow$ bias, greatly $\downarrow$ variance

3. Linear Algebra Foundations

Eigenvalue Equation

$$\mathbf{A}\mathbf{v} = \lambda \mathbf{v}$$

λ = eigenvalue (how much the vector is scaled)
 $\mathbf{v}$ = eigenvector (direction unchanged by $\mathbf{A}$)

Eigendecomposition (symmetric $\mathbf{A}$)

$$\mathbf{A} = \mathbf{Q} \mathbf{\Lambda} \mathbf{Q}^\top$$

$\mathbf{Q}$: columns = eigenvectors (orthonormal)

$$\mathbf{\Lambda} = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_d)$$

Singular Value Decomposition (SVD)

For any $\mathbf{A} \in \mathbb{R}^{m \times n}$:

$$\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top$$

- $\mathbf{U} \in \mathbb{R}^{m \times m}$: left singular vectors (orthogonal)
- $\mathbf{\Sigma} \in \mathbb{R}^{m \times n}$: diagonal, $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$
- $\mathbf{V} \in \mathbb{R}^{n \times n}$: right singular vectors (orthogonal)

Key facts:

- $\sigma_i = \sqrt{\lambda_i(\mathbf{A}^\top \mathbf{A})}$
- $\text{rank}(\mathbf{A})$ = number of non-zero σ_i
- Works for *any* matrix (not just square/symmetric)

Best Rank- k Approximation (Eckart-Young)

$$\mathbf{A}_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$$

Optimal in Frobenius norm. Applications: image compression, denoising, recommendation systems.

Choosing k : look for “elbow” in the σ plot.

4. Principal Component Analysis (PCA)

PCA Algorithm – Step by Step

1. **Center:** $\tilde{\mathbf{x}}_i = \mathbf{x}_i - \bar{\mathbf{x}}, \quad \bar{\mathbf{x}} = \frac{1}{n} \sum_i \mathbf{x}_i$
2. **Covariance matrix:**

$$\Sigma = \frac{1}{n} \tilde{\mathbf{X}}^\top \tilde{\mathbf{X}} \in \mathbb{R}^{d \times d}$$

3. **Eigen-decompose** Σ : get $\lambda_j, \mathbf{v}_j$
4. **Sort** by $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d$
5. **Select top** d' : $\mathbf{U} = [\mathbf{v}_1, \dots, \mathbf{v}_{d'}]$
6. **Project:** $\mathbf{x}'_i = \mathbf{U}^\top \tilde{\mathbf{x}}_i \in \mathbb{R}^{d'}$

Covariance Matrix Entry

$$\Sigma_{jk} = \frac{1}{n} \sum_{i=1}^n (x_{ij} - \bar{x}_j)(x_{ik} - \bar{x}_k)$$

Diagonal = variance of feature j . Off-diagonal = covariance. Always symmetric, positive semi-definite.

Reconstruction & Variance Explained

Reconstruct from projection:

$$\hat{\mathbf{x}}_i = \mathbf{U} \mathbf{x}'_i + \bar{\mathbf{x}}$$

Fraction of total variance captured:

$$\text{Var. Explained} = \frac{\sum_{j=1}^{d'} \lambda_j}{\sum_{j=1}^d \lambda_j} \quad (\text{aim for } \geq 90\%)$$

PCA via SVD Connection

Given $\tilde{\mathbf{X}} = \mathbf{U}_X \Sigma_X \mathbf{V}_X^\top$:

- Right singular vectors $\mathbf{V}_X$ = principal components
- $\lambda_j = \sigma_j^2 / n$

SVD is numerically more stable than explicit covariance.

PCA Warnings

- **Always standardize** (zero mean, unit variance) before PCA
- PCA is **linear only** – cannot capture non-linear structure
- For non-linear: use **t-SNE** or **UMAP**

5. Bayesian Decision Theory

Bayes' Theorem

$$P(\omega_j | \mathbf{x}) = \frac{P(\mathbf{x} | \omega_j) P(\omega_j)}{P(\mathbf{x})}$$

$$\text{Posterior} = \frac{\text{Likelihood} \times \text{Prior}}{\text{Evidence}}$$

Bayesian Decision Rule (MAP)

$$\hat{y} = \arg \max_j P(\mathbf{x} | \omega_j) P(\omega_j)$$

$P(\mathbf{x})$ is the same for all classes, so we drop it. This **minimises** probability of error.

Univariate Gaussian PDF

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

μ = mean (centre), σ^2 = variance (spread)

Multivariate Gaussian PDF

$$p(\mathbf{x}) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu})\right)$$

$\boldsymbol{\mu}$: mean vector, Σ : covariance matrix

Decision Boundary Shape

- **Same** $\Sigma_1 = \Sigma_2 \Rightarrow$ **linear** boundary (hyperplane)
- **Different** $\Sigma_1 \neq \Sigma_2 \Rightarrow$ **quadratic** boundary (curve/ellipse)

6. MLE & Gaussian Mixture Models

MLE for Gaussian (i.i.d. data)

Log-likelihood:

$$\ell = -\frac{n}{2} \ln(2\pi) - \frac{n}{2} \ln \sigma^2 - \frac{1}{2\sigma^2} \sum_{i=1}^n (x_i - \mu)^2$$

Solutions (set derivative to zero):

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^n x_i \quad \hat{\sigma}^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{\mu})^2$$

Gaussian Mixture Model (GMM)

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \Sigma_k)$$

π_k = mixing weight, $\sum_k \pi_k = 1$, $\pi_k \geq 0$.

EM Algorithm for GMM

E-Step – compute responsibilities:

$$\gamma_{ik} = \frac{\pi_k \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_j, \Sigma_j)}$$

M-Step – update parameters ($N_k = \sum_i \gamma_{ik}$):

$$\boldsymbol{\mu}_k = \frac{1}{N_k} \sum_i \gamma_{ik} \mathbf{x}_i$$

$$\Sigma_k = \frac{1}{N_k} \sum_i \gamma_{ik} (\mathbf{x}_i - \boldsymbol{\mu}_k)(\mathbf{x}_i - \boldsymbol{\mu}_k)^\top$$

$$\pi_k = \frac{N_k}{n}$$

Repeat E and M steps until convergence.

EM Caveats

- Converges to **local** maximum only
- Run multiple times with different initialisations
- Must choose K (use BIC/AIC/cross-validation)

K-Means Algorithm (Hard Clustering)

1. **Init** K centroids $\mu_1, \dots, \mu_K$
 2. **Assign**: $c_i = \arg \min_k \|\mathbf{x}_i - \mu_k\|^2$
 3. **Update**: $\mu_k = \frac{1}{|C_k|} \sum_{i \in C_k} \mathbf{x}_i$
 4. Repeat steps 2–3 until stable
- K-Means = GMM with equal spherical covariances (hard assignments).

7. Classification Algorithms

Naive Bayes

Assumes features independent given class:

$$P(\mathbf{x} | \omega_j) = \prod_{i=1}^d P(x_i | \omega_j)$$

Decision rule:

$$\hat{y} = \arg \max_j P(\omega_j) \prod_{i=1}^d P(x_i | \omega_j)$$

Fast, good for high-dim data (e.g., text). Independence assumption often violated.

Logistic Regression (Binary)

$$P(y = 1 | \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}}$$

Sigmoid properties:

- Output in $(0, 1)$ – interpreted as probability
- $\sigma(0) = 0.5$
- $\sigma(-z) = 1 - \sigma(z)$
- $\sigma'(z) = \sigma(z)(1 - \sigma(z))$

Cross-Entropy Loss (Logistic Regression)

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^n \left[y_i \ln \hat{p}_i + (1 - y_i) \ln(1 - \hat{p}_i) \right]$$

where $\hat{p}_i = \sigma(\mathbf{w}^\top \mathbf{x}_i + b)$. Minimise via gradient descent.

Softmax (Multi-Class, K classes)

$$P(y = k | \mathbf{x}) = \frac{e^{\mathbf{w}_k^\top \mathbf{x}}}{\sum_{j=1}^K e^{\mathbf{w}_j^\top \mathbf{x}}}$$

Outputs sum to 1. Loss: categorical cross-entropy.

Loss Functions Summary

Loss	Formula	Used for
0–1	$\mathbf{1}[\hat{y} \neq y]$	Ideal, non-diff.
MSE	$\frac{1}{n} \sum (y_i - \hat{y}_i)^2$	Regression
Hinge	$\max(0, 1 - y \cdot f(x))$	SVM
Log	$\ln(1 + e^{-y \cdot f(x)})$	Log. Reg.
X-entropy	$-\sum y_k \ln \hat{p}_k$	Multi-class

8. Support Vector Machines (SVM)

Core Idea

Find the hyperplane with **maximum margin** between classes.
Margin = $\frac{2}{\|\mathbf{w}\|}$. Maximising margin $\Leftrightarrow$ minimising $\|\mathbf{w}\|^2$.
Support vectors: points closest to the boundary.

Hard-Margin SVM (linearly separable data)

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 \quad \forall i$$

Soft-Margin SVM (slack variables $\xi_i \geq 0$)

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$$

$$\text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

Large C : fewer errors allowed (may overfit).
Small C : wider margin, more errors allowed (may underfit).

Kernel Functions

$K(\mathbf{x}_i, \mathbf{x}_j) = \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle$ (inner product in high-dim space)

Kernel	Formula
Linear	$\mathbf{x}_i^\top \mathbf{x}_j$
Polynomial (deg p)	$(\mathbf{x}_i^\top \mathbf{x}_j + c)^p$
RBF / Gaussian	$\exp\left(-\frac{\ \mathbf{x}_i - \mathbf{x}_j\ ^2}{2\sigma^2}\right)$

Kernel trick: compute similarity in high-dim space **without** explicit mapping.

9. Neural Networks

Single Neuron (Perceptron)

$$\hat{y} = \text{activation}\left(\sum_{j=1}^d w_j x_j + b\right) = \text{activation}(\mathbf{w}^\top \mathbf{x} + b)$$

Perceptron Learning Rule (Classification)

For each *misclassified* point ($y_i \in \{-1, +1\}$):

$$\mathbf{w} \leftarrow \mathbf{w} + \eta y_i \mathbf{x}_i$$

Only updates on mistakes. Converges if data is linearly separable.

Delta Rule (Regression – gradient on MSE)

$$\mathbf{w} \leftarrow \mathbf{w} + \eta (y_i - \hat{y}_i) \mathbf{x}_i$$

Updates proportional to error ($y_i - \hat{y}_i$).

Activation Functions

Name	Formula	Notes
Step	$\mathbf{1}[z \geq 0]$	Not differentiable
Sigmoid	$\frac{1}{1 + e^{-z}}$	Output in $(0, 1)$
Tanh	$\frac{e^z - e^{-z}}{e^z + e^{-z}}$	Output in $(-1, 1)$
ReLU	$\max(0, z)$	Most popular

MLP (Multi-Layer Perceptron)

Architecture: Input $\rightarrow$ Hidden layer(s) $\rightarrow$ Output

Each layer: $\mathbf{h} = \text{activation}(\mathbf{W}\mathbf{x} + \mathbf{b})$

Universal Approximation Theorem: one hidden layer + nonlinear activation can approximate *any* continuous function.

XOR is not linearly separable $\Rightarrow$ needs MLP (or kernel trick).

10. Metrics & Practical ML

Classification Metrics

Metric	Formula
Accuracy	$\frac{TP + TN}{TP + TN + FP + FN}$
Precision	$\frac{TP}{TP + FP}$
Recall	$\frac{TP}{TP + FN}$
F1 Score	$2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$

Confusion Matrix

	Predicted +	Predicted –
Actual +	TP	FN
Actual –	FP	TN

Data Normalization

Min-Max Scaling (to $[0, 1]$):

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Z-score Standardization (zero mean, unit variance):

$$x' = \frac{x - \mu}{\sigma}$$

Data Splits & Cross-Validation

Train / Validation / Test: 60–70% / 15–20% / 15–20%

Golden rule: never use the test set during training or tuning!

k -Fold Cross-Validation:

1. Split training data into k equal folds
2. Train on $k - 1$ folds, validate on the remaining 1
3. Repeat k times; average the scores

Typical: $k = 5$ or $k = 10$.